EECE 7205 In-Class Work: Binary Search Trees

Karishma Premnath

Height is counted in edges: a single node has height 0, and the tree built below has height 3.

Problem 1: Building the tree (insert)

Case 3: Insert a larger key (insert 70)

(a) Rules and explanation

Since $70 > 50$, rule **I3** says go right: any key larger than a node belongs in its right subtree. The right child of 50 is NULL, so rule **I1** attaches 70 there as a leaf. **One comparison.**

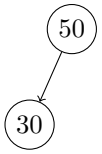
At node	Comparison	Rule	Action
50	$70 > 50$	I3	go right
right child of 50	NULL	I1	attach 70 here

(b) Pseudocode for this case

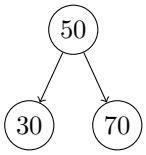
INSERT-LARGER-KEY(*key*)

```
1: newNode ← new node holding key, both children NULL
2: cur ← root
3: if key > cur.key then
4:     if cur.right = NULL then
5:         cur.right ← newNode
6:         return
7:     end if
8:     cur ← cur.right
9: end if
```

(c) Tree before and after



Before: {50, 30}

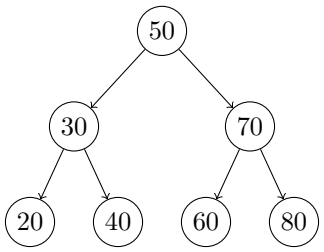


After: 70 is the right child of 50

Case 4: Insert deep in the tree, turning left and right

Inserting 20, 40, 60 and 80 first

Insert	Nodes compared	Rules	Lands as
20	50, 30	I2, I2, I1	left child of 30
40	50, 30	I2, I3, I1	right child of 30
60	50, 70	I3, I2, I1	left child of 70
80	50, 70	I3, I3, I1	right child of 70



(a) Inserting 35: rules and explanation

At node	Comparison	Rule	Action
50	$35 < 50$	I2	go left
30	$35 > 30$	I3	go right
40	$35 < 40$	I2	go left
left child of 40	NULL	I1	attach 35 here

Nodes compared, in order: **50, 30, 40**. That is **3 comparisons**, and 35 becomes the left child of 40 at depth 3.

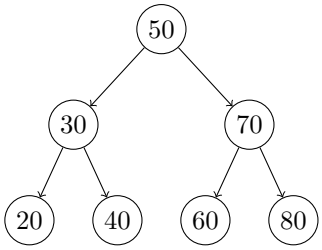
35 goes left at 50 and right at 30 because every comparison uses only the node you are standing on. Left at 50 gives $35 < 50$ and right at 30 gives $35 > 30$, which leaves 35 between 30 and 50. At 40 the window narrows to between 30 and 40. The walk stops at the first NULL pointer inside that window, and each level costs one comparison.

(b) Pseudocode: the loop that walks down

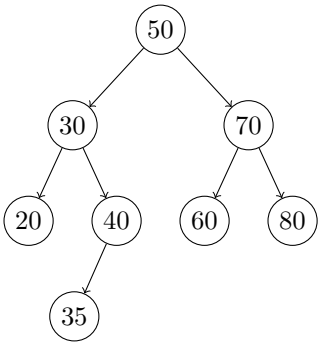
INSERT(key)

```
1:  newNode ← new node holding key, both children NULL
2:  cur ← root
3:  while true do
4:      if key < cur.key then
5:          if cur.left = NULL then
6:              cur.left ← newNode
7:              return
8:          end if
9:          cur ← cur.left
10:     else if key > cur.key then
11:         if cur.right = NULL then
12:             cur.right ← newNode
13:             return
14:         end if
15:         cur ← cur.right
16:     else
17:         return
18:     end if
19: end while
```

(c) Tree before and after inserting 35



Before: seven keys, height 2



After: 35 is the left child of 40

Case 5: Insert a key that is already in the tree (insert 40 again)

Finishing the tree with 45, 65 and 75

Insert	Nodes compared	Rules	Lands as
45	50, 30, 40	I2, I3, I3, I1	right child of 40
65	50, 70, 60	I3, I2, I3, I1	right child of 60
75	50, 70, 80	I3, I3, I2, I1	left child of 80

(a) Inserting 40 a second time: rules and explanation

At node	Comparison	Rule	Action
50	$40 < 50$	I2	go left
30	$40 > 30$	I3	go right
40	$40 = 40$	I4	duplicate, nothing is inserted

The walk goes left at 50 and right at 30, landing on the node already holding 40. The keys are equal, so rule **I4** applies and no node is created. **3 comparisons**, the same as a successful search for 40.

A binary search tree must not store the key twice. The BST property is strict: left subtree smaller, right subtree larger. A key equal to the node is neither, so no side of it is legal. A second copy would break search, delete and inorder traversal. Search stops at the first match under rule S2 and never reaches the lower copy. Delete removes one copy and leaves the other, so a search for 40 still finds it right after deleting. Inorder traversal would print 40 twice instead of once.

To record repeats, add a *count* field and increment it. That way each key still has one node.

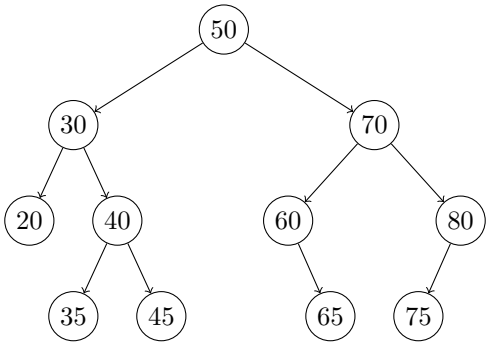
(b) Pseudocode for this case

fragment of INSERT(key), the walk that meets an existing key

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if key < cur.key then
4:          cur ← cur.left
5:      else if key > cur.key then
6:          cur ← cur.right
7:      else
8:          return
9:      end if
10: end while
```

This shows only the walk and the I4 exit. The branches that attach a new node are left out here; they are in part (h).

(c) Final tree of `arr` (all 11 keys)



The height of this tree is 3, and the deepest inserts (35, 45, 65 and 75) each need 3 comparisons. It is also the tree after the second insert of 40: rule I4 creates no node and moves no pointer, leaving the drawing unchanged.

Case 6: Insert keys that arrive in sorted order (20, 30, 40, 50, 60)

(a) Rule(s) and explanation

Insert	Nodes compared	Rules	Lands as	Comparisons
20	none	I1	root	0
30	20	I3, I1	right child of 20	1
40	20, 30	I3, I3, I1	right child of 30	2
50	20, 30, 40	I3, I3, I3, I1	right child of 40	3
60	20, 30, 40, 50	I3, I3, I3, I3, I1	right child of 50	4

Every insert after the first uses **I3** at each node, then **I1** to attach. **I2 never applies:** each key is larger than everything already stored, so the walk turns right at every node.

Height 4 for only five keys. The tree in Case 5 holds eleven keys and its height is 3. Sorted input gives height $n-1$ instead of about $\log_2 n$. Building it cost $0+1+2+3+4=10$ comparisons, which is $n(n-1)/2$.

Every left pointer is NULL, so this is really a linked list. Each node wastes one pointer and a search costs $O(n)$. The two fixes are to insert the median key first, or to use a self-balancing tree such as AVL or red-black.

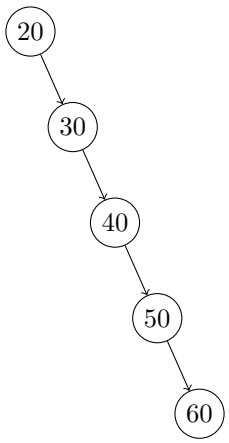
(b) Pseudocode BUILD-FROM-ARRAY(*data*, *n*)

BUILD-FROM-ARRAY(*data*, *n*)

```
1: root ← NULL
2: for i ← 0 to n − 1 do
3:     INSERT(data[i])
4: end for
5: return root
```

The loop never inspects the order of *data*.

(c) Tree



Height 4. Every node except 60 has exactly one child, always on the right.

(g) Complete table for inserting all of `arr`

Step	Insert	Nodes compared (in order)	Rule(s)	Parent	Left / Right
1	50	none	I1	-	root
2	30	50	I2, I1	50	Left
3	70	50	I3, I1	50	Right
4	20	50, 30	I2, I2, I1	30	Left
5	40	50, 30	I2, I3, I1	30	Right
6	60	50, 70	I3, I2, I1	70	Left
7	80	50, 70	I3, I3, I1	70	Right
8	35	50, 30, 40	I2, I3, I2, I1	40	Left
9	45	50, 30, 40	I2, I3, I3, I1	40	Right
10	65	50, 70, 60	I3, I2, I3, I1	60	Right
11	75	50, 70, 80	I3, I3, I2, I1	80	Left

I1 ends every row. It never picks a direction; I2 and I3 do that. I4 appears nowhere, because `arr` holds no repeated key; Case 5 had to insert 40 a second time by hand to exercise it. The number of nodes compared equals the depth the key lands at. The eleven inserts cost 22 comparisons altogether.

(h) One complete INSERT, with height and deepest insert

```
INSERT(key)
 1: newNode ← new node holding key, left = right = NULL
 2: if root = NULL then
 3:     root ← newNode
 4:     return
 5: end if
 6: cur ← root
 7: while true do
 8:     if key < cur.key then
 9:         if cur.left = NULL then
10:             cur.left ← newNode
11:             return
12:         end if
13:         cur ← cur.left
14:     else if key > cur.key then
15:         if cur.right = NULL then
16:             cur.right ← newNode
17:             return
18:         end if
19:         cur ← cur.right
20:     else
21:         return
22:     end if
23: end while
```

All five cases sit in one procedure: lines 2 to 5 are Case 1, lines 8 to 13 Cases 2 and 4 on the left, lines 14 to 19 Cases 3 and 4 on the right, lines 20 to 21 Case 5. Rule I1 appears twice because an empty spot means two things, the empty tree at line 2 and an empty child pointer at lines 9 and 15.

Height of the final tree: 3. Comparisons for the deepest insert: 3.

The deepest keys 35, 45, 65 and 75 each land at depth 3 after three comparisons. An insert that lands at depth d costs d comparisons. In Case 6 the keys land deeper and deeper, the last one at depth $n - 1$.

Problem 2: Searching the tree

All searches use the final tree of Problem 1 (all 11 keys). Shaded nodes mark the search path; a dashed box marks the NULL pointer where an unsuccessful search stops.

Case 3: The target is in the right subtree, path turns (search 65)

(a) Rule at each node and explanation

At node	Comparison	Rule	Move
50	$65 > 50$	S4	go right
70	$65 < 70$	S3	go left
60	$65 > 60$	S4	go right
65	$65 = 65$	S2	found

Nodes compared: **50, 70, 60, 65**. **Found after 4 comparisons.** The path turns right, then left, then right, because each comparison looks only at the node it is on.

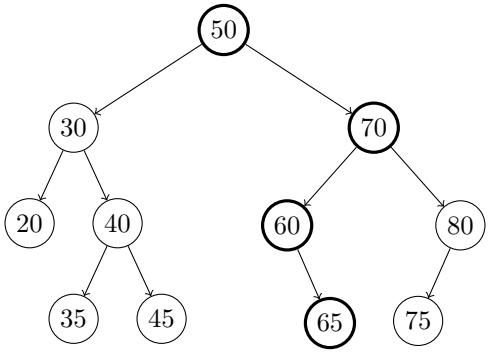
(b) Pseudocode for this case

SEARCH(*target*)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then
4:          return FOUND
5:      else if target < cur.key then
6:          cur ← cur.left
7:      else
8:          cur ← cur.right
9:      end if
10: end while
11: return NOT FOUND
```

For target 65 the branches run in the order S4, S3, S4, S2, and line 4 returns FOUND at the fourth node.

(c) Tree with the search path marked



Search path: $50 \to 70 \to 60 \to 65$.

Case 4: The target is not in the tree (search 33)

(a) Rule at each node and explanation

At node	Comparison	Rule	Move
50	$33 < 50$	S3	go left
30	$33 > 30$	S4	go right
40	$33 < 40$	S3	go left
35	$33 < 35$	S3	go left
left child of 35	NULL	S1	not found

Nodes compared: **50, 30, 40, 35**. **Not found after 4 comparisons.**

The search stops at the **left pointer of 35**, which is NULL, and rule S1 reports failure.

If 33 were inserted, its parent would be **35** and it would be the **left** child. A failed search ends at exactly the pointer an insert would fill, so SEARCH and INSERT walk the same path.

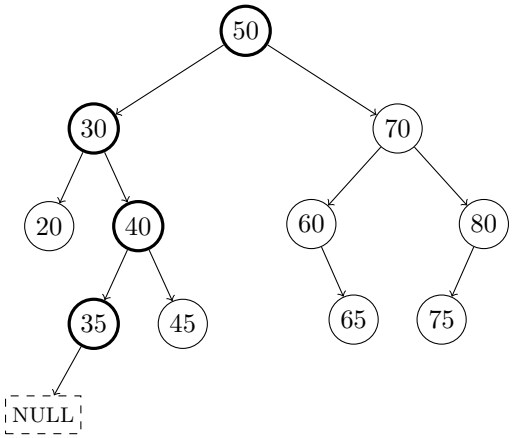
(b) Pseudocode for this case

SEARCH(*target*)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then
4:          return FOUND
5:      else if target < cur.key then
6:          cur ← cur.left
7:      else
8:          cur ← cur.right
9:      end if
10: end while
11: return NOT FOUND
```

For target 33 the branches run S3, S4, S3, S3. After the fourth pass *cur* becomes the left child of 35, which is NULL, so the test at line 2 fails and line 11 returns NOT FOUND. S2 never runs.

(c) Tree with the search path marked



Search path: $50 \to 30 \to 40 \to 35 \to$ NULL.

Case 5: The target is larger than every key (search 90)

(a) Rule at each node and explanation

At node	Comparison	Rule	Move
50	$90 > 50$	S4	go right
70	$90 > 70$	S4	go right
80	$90 > 80$	S4	go right
right child of 80	NULL	S1	not found

Nodes compared: **50, 70, 80**. **Not found after 3 comparisons.** S4 fires at every node and S3 never fires, so the walk stays on the rightmost branch and falls off the end. That is fewer comparisons than Case 4, because the right branch is shorter than the path down to 35.

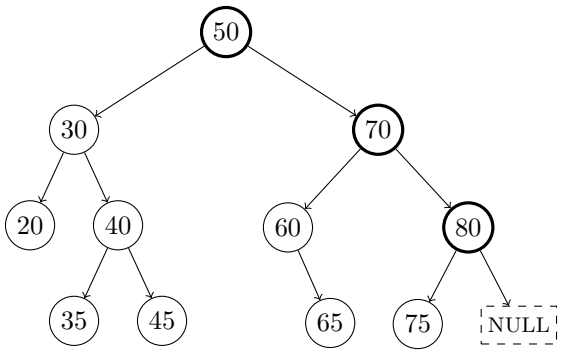
(b) Pseudocode for this case

SEARCH(*target*)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then
4:          return FOUND
5:      else if target < cur.key then
6:          cur ← cur.left
7:      else
8:          cur ← cur.right
9:      end if
10: end while
11: return NOT FOUND
```

For target 90 only the S4 branch at lines 7 to 8 ever runs. After the third pass *cur* becomes the right child of 80, which is NULL, and line 11 returns NOT FOUND.

(c) Tree with the search path marked



Search path: $50 \to 70 \to 80 \to$ NULL.

Case 6: Find the minimum and the maximum key

(a) Rule and explanation

No target is given, so S2, S3 and S4 never run. Only the pointers are followed.

Follow left pointers for the minimum and right pointers for the maximum. Smaller keys always sit on the left and larger keys always on the right, so the leftmost node is the smallest and the rightmost node is the largest.

The walk stops when *cur.left* is NULL for the minimum, and when *cur.right* is NULL for the maximum. Nothing below 20 can exist, because 20's left subtree is empty.

Minimum = 20 via $50 \to 30 \to 20$. **Maximum** = 80 via $50 \to 70 \to 80$. Three nodes visited on each path.

(b) Pseudocode FIND-MIN and FIND-MAX

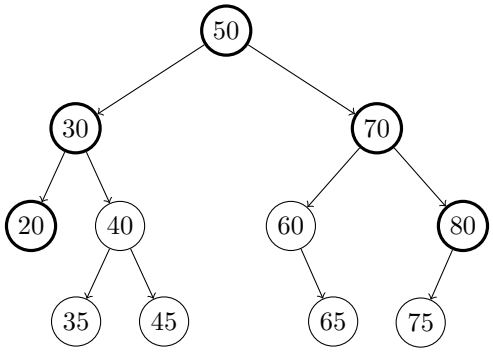
FIND-MIN(*root*)

```
1: if root = NULL then
2:     return NULL
3: end if
4: cur ← root
5: while cur.left ≠ NULL do
6:     cur ← cur.left
7: end while
8: return cur.key
```

FIND-MAX(*root*)

```
1: if root = NULL then
2:     return NULL
3: end if
4: cur ← root
5: while cur.right ≠ NULL do
6:     cur ← cur.right
7: end while
8: return cur.key
```

(c) Tree with both paths marked



Left spine $50 \to 30 \to 20$ gives the minimum; right spine $50 \to 70 \to 80$ gives the maximum.

Problem 2 summary table

Case	Target	Nodes compared (in order)	Rules	Result	Comparisons
1	50	50	S2	found	1
2	45	50, 30, 40, 45	S3, S4, S4, S2	found	4
3	65	50, 70, 60, 65	S4, S3, S4, S2	found	4
4	33	50, 30, 40, 35	S3, S4, S3, S3, S1	not found	4
5	90	50, 70, 80	S4, S4, S4, S1	not found	3
6	min	50, 30, 20	leftmost path	20	0
6	max	50, 70, 80	rightmost path	80	0

Case 6 makes no key comparisons. Each path is two pointer steps, ending at a NULL child.

(g) The largest number of comparisons any search can need

4 comparisons. The tree has height 3, and a search makes one comparison per level, so the worst case is $h + 1 = 4$: the number of nodes on the longest root-to-leaf path. Cases 2, 3 and 4 all hit that bound, two of them successfully and one not.

What matters is the height, not the number of keys. These 11 keys in a tree of height 3 need at most 4 comparisons. The sorted tree of Problem 1 Case 6 has $h = n - 1$, so it needs n.

Problem 3: Deleting from the tree

Case 3: Delete a node with only a right child (delete 60)

(a) Rule(s) and explanation

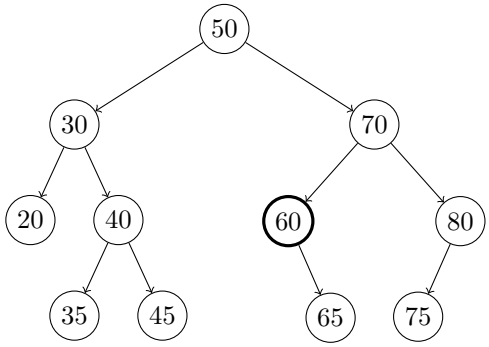
Search 50 → 70 → 60 (S4, S3, S2). 60 has **one child**, the right child 65. **Rule D2** replaces 60 by 65. The left pointer of 70 now points at 65. Nothing else moves. 65 is already bigger than everything left of 60 and smaller than 70, so it fits that spot.

(b) Pseudocode for this case

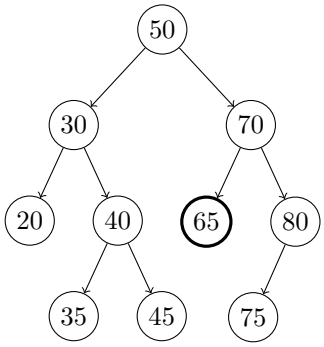
fragment of DELETE(*node*, *key*), the found branch

```
1: else if node.left = NULL then
2:     return node.right
```

(c) Tree before and after



Before: 60 has only a right child



After: 65 takes 60's place

Case 4: Delete a node with only a left child (delete 80)

(a) Rule(s) and explanation

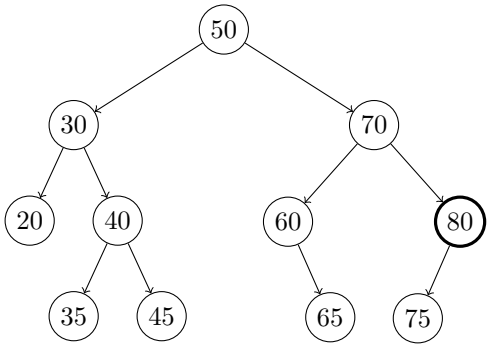
Search 50 → 70 → 80 (S4, S4, S2). 80 has **one child**, the left child 75. **Rule D2** again, mirrored: 80 is replaced by 75, and the right pointer of 70 points at 75.

(b) Pseudocode for this case

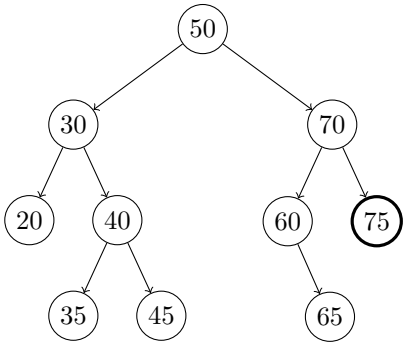
fragment of DELETE(*node*, *key*), the found branch

```
1: else if node.right = NULL then
2:     return node.left
```

(c) Tree before and after



Before: 80 has only a left child



After: 75 takes 80's place

Case 5: Delete a node with two children, successor is its right child (delete 40)

(a) Rule(s) and explanation

Search $50 \to 30 \to 40$ (S3, S4, S2). 40 has **two children**, 35 and 45, so **D3** applies.

To find the inorder successor of 40, go right once to 45, then left as far as possible. 45 has no left child. The walk stops at once, making **45** the successor.

It is 40's own right child here because the successor is the leftmost node of the right subtree. 45 has no left subtree, so 45 is itself the leftmost node.

Copy 45 into the node, then delete the old 45 from the right subtree. That 45 is a leaf, making the second step **D1**. Rules applied: **D3 then D1**.

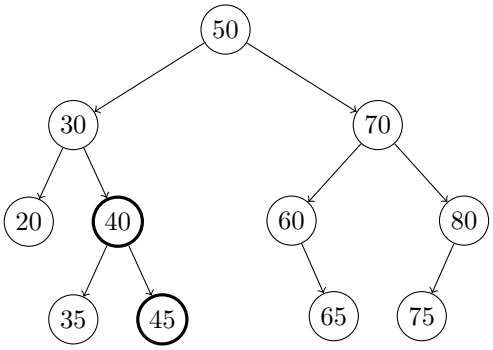
(b) Pseudocode for this case

fragment of DELETE(*node*, *key*), the found branch

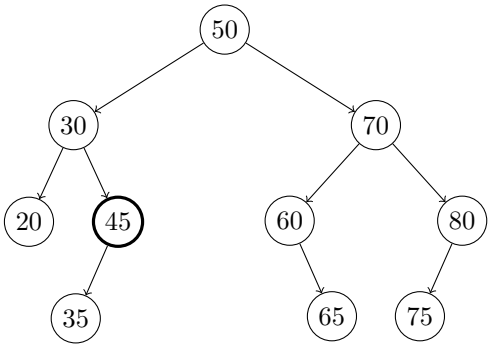
```
1: else
2:     succ ← FIND-MIN-NODE(node.right)
3:     node.key ← succ.key
4:     node.right ← DELETE(node.right, succ.key)
```

FIND-MIN-NODE is the walk of FIND-MIN from Problem 2, stopping at the leftmost *node* instead of returning its key, so that *succ.key* can be read from it.

(c) Tree before and after



Before: 40 has two children, successor 45



After: 45 copied up, old 45 removed by D1

Case 6: Delete the root, whose successor has a child of its own (delete 50)

(a) Rule(s) and explanation

50 is the root and has **two children**, so **D3** applies.

To find the successor of 50, go right once to 70, then left as far as possible: 70 has a left child 60, and 60 has no left child, which makes **60** the successor. Unlike Case 5, the successor is not 50's own right child, and it carries a right child of its own, 65.

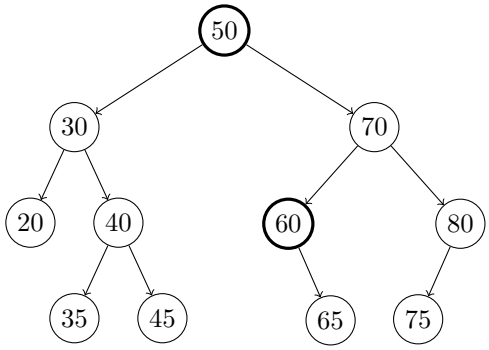
After copying 60 into the root, the old 60 has **only a right child**, and removing it uses **Case 3's rule, D2**: 60 is replaced by 65. Rules applied: **D3 then D2**.

(b) Pseudocode for this case

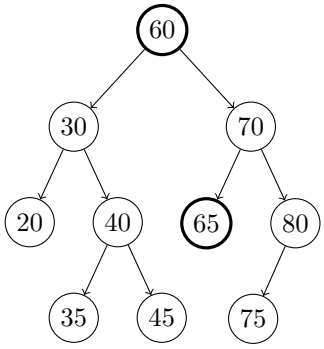
fragment of DELETE(*node*, *key*), the found branch

```
1: else
2:     succ ← FIND-MIN-NODE(node.right)
3:     node.key ← succ.key
4:     node.right ← DELETE(node.right, succ.key)
```

(c) Tree before and after



Before: root 50, successor 60



After: 60 at the root, 65 takes 60's old place

Problem 3 summary table

Case	Delete	Children of the node	Successor	Rules applied	Level order after
1	33	not found	-	S1	50 30 70 20 40 60 80 35 45 65 75
2	20	none	-	D1	50 30 70 40 60 80 35 45 65 75
3	60	one (right)	-	D2	50 30 70 20 40 65 80 35 45 75
4	80	one (left)	-	D2	50 30 70 20 40 60 75 35 45 65
5	40	two	45	D3, D1	50 30 70 20 45 60 80 35 65 75
6	50	two	60	D3, D2	60 30 70 20 40 65 80 35 45 75

(g) One complete DELETE(node, key)

DELETE(*node*, *key*), returns the new root of this subtree

```
1: if node = NULL then
2:     return NULL
3: end if
4: if key < node.key then
5:     node.left ← DELETE(node.left, key)
6: else if key > node.key then
7:     node.right ← DELETE(node.right, key)
8: else
9:     if node.left = NULL and node.right = NULL then
10:        return NULL
11:    else if node.left = NULL then
12:        return node.right
13:    else if node.right = NULL then
14:        return node.left
15:    else
16:        succ ← FIND-MIN-NODE(node.right)
17:        node.key ← succ.key
18:        node.right ← DELETE(node.right, succ.key)
19:    end if
20: end if
21: return node
```

The caller writes $root \leftarrow \text{DELETE}(root, key)$, which is how Case 6 replaces the root without needing a special case.

(h) Why copy the inorder successor instead of moving the deepest node

Whatever goes into the empty spot has to be bigger than every key on the left and smaller than every key on the right. Only one key does that: the smallest key in the right subtree, which is the inorder successor. The deepest node is just whatever sits at the bottom, so it would usually break $left < node < right$. A general binary tree has no order to keep, so there the deepest node is fine.

The successor also has no left child, so deleting it is always D1 or D2, never D3.